TASK 2 – Read and Print Data Correctly (4 Marks)

Example C program with input validation, rollover handling, and correctly formatted currency:

File: Task02/billing_input.c

```
#include <stdio.h>
#include <stdint.h>

#define FREE_UNITS 100u
#define SECOND_SLAB_LIMIT 200u
#define SECOND_SLAB_PAISE 225u
#define THIRD_SLAB_PAISE 450u
#define FIXED_CHARGE_PAISE 3000LL

static int64_t bill_for_units(uint32_t units)
{
    int64_t energy = 0;
    if (units > FREE_UNITS) {
        uint32_t second = units < SECOND_SLAB_LIMIT
                        ? units - FREE_UNITS
                        : SECOND_SLAB_LIMIT - FREE_UNITS;
        energy += (int64_t)second * SECOND_SLAB_PAISE;
    }
    if (units > SECOND_SLAB_LIMIT) {
        energy += (int64_t)(units - SECOND_SLAB_LIMIT) * THIRD_SLAB_PAISE;
    }
    return FIXED_CHARGE_PAISE + energy;
}

static void print_money(int64_t paise)
{
    printf("Rs %lld.%02lld\\n",
           (long long)(paise / 100), (long long)(paise % 100));
}

int main(void)
{
    unsigned int prev, curr;
    uint32_t units;
    int64_t bill;

    printf("Enter previous reading (0-65535): ");
    if (scanf("%u", &prev) != 1 || prev > 65535u) {
        fprintf(stderr, "Invalid previous reading.\\n");
        return 1;
    }
    printf("Enter current reading (0-65535): ");
    if (scanf("%u", &curr) != 1 || curr > 65535u) {
        fprintf(stderr, "Invalid current reading.\\n");
        return 1;
    }

    units = (uint16_t)((uint16_t)curr - (uint16_t)prev);
    bill = bill_for_units(units);

    printf("Units consumed: %u\\n", units);
    printf("Fixed charge: Rs 30.00\\n");
    printf("Total bill: ");
    print_money(bill);
    return 0;
}
```

Format specifiers

A wrong format specifier can cause incorrect output or undefined behaviour. For example, passing a long to printf with %d is a type mismatch; use %ld for long. scanf returns the number of successful assignments, so check it before using input. For stronger recovery from invalid lines, use fgets() to read a bounded line and strtol() to parse and validate the whole token. Reading a line of length L takes O(L) time and O(B) buffer space for a buffer of B characters. fgets/strtol makes it easier to discard an invalid line and prompt again; scanf can leave unwanted characters in the input stream unless they are consumed.

Sample test

Input: previous reading = 1000, current reading = 1250

Output: Units consumed: 250; fixed charge: Rs 30.00; total bill: Rs 480.00

Calculation: 100 free units + 100 × Rs 2.25 + 50 × Rs 4.50 + Rs 30.00 = Rs 480.00.